

NOTES

Advance Pandas



Pandas Advanced

✓ Data Aggregation and Grouping in Pandas

In data analysis, we often need to summarize or organize data into smaller, meaningful chunks. Data Aggregation and Grouping in Pandas help us do just that. These tools allow us to group data by certain categories (e.g., regions, departments) and then calculate statistics like averages, totals, counts, or more advanced metrics. Let's break it down in simple terms and cover everything step-by-step.

✓ 1.Data Aggregation

a. What is Data Aggregation ?

- Data aggregation is the process of summarizing or transforming data into a more meaningful form.
- This is commonly used in data analysis to calculate metrics such as averages, totals, counts, or other statistics for specific groups within a dataset.
- Aggregation helps uncover insights from large datasets, making them easier to interpret.

b. Why Use Data Aggregation?

Data aggregation is important due to the following reasons:

- **Makes Data Easier to Understand:** Large datasets can be hard to work with. Aggregation simplifies the data by summarizing it into clear, meaningful numbers. For example, instead of looking at every sale made, you can focus on the total sales.
- **Perform Simple Calculations:**
Aggregation lets you do calculations on grouped data, such as:
 - Sum: Add up all the numbers in a group (e.g., total income).
 - Average: Find the middle value in a group (e.g., average customer spending).
 - Count: See how many items are in a group (e.g., the number of sales).
 - Other Calculations: Use more advanced math, like finding the highest or lowest value.
- **Spot Patterns or Trends in Smaller Groups:**
Aggregation helps you find patterns or trends in specific parts of your data. For example:

- Sales Trend by Region: See which areas are doing better or worse in sales.
- Average Salary by Department: Compare how salaries differ between teams to find gaps or make plans.

By focusing on smaller groups, you can uncover useful insights that might be hidden in the overall data.

In short, data aggregation makes it easier to work with data, find important patterns, and make better decisions.

✓ c. Examples For Data Aggregation

✓ 1.Simple Aggregation with a Single Function

```
import pandas as pd

# Sample DataFrame
data = {
    'Category': ['A', 'A', 'B', 'B', 'C', 'C'],
    'Values': [10, 20, 15, 25, 30, 35],
    'Score': [1, 2, 3, 4, 5, 6]
}
df = pd.DataFrame(data)

# Aggregation: Sum of 'Values' grouped by 'Category'
result = df.groupby('Category')['Values'].sum()
print(result)
```

Output:

```
Category
A      30
B      40
C      65
Name: Values, dtype: int64
```

Explanation:

- The `groupby` function groups data by **Category** (A, B, C).
- The `sum` function adds up the **Values** for each group:
 - **A**: $10 + 20 = 30$
 - **B**: $15 + 25 = 40$
 - **C**: $30 + 35 = 65$

✓ 2. Multiple Aggregations on a Single Column

```
import pandas as pd

# Sample Data
data = {'Category': ['A', 'B', 'A', 'C', 'B', 'C'],
        'Values': [10, 15, 20, 30, 25, 35]}

# Create DataFrame
df = pd.DataFrame(data)

# Aggregation: Applying 'sum' and 'mean' to 'Values'
result = df.groupby('Category')['Values'].agg(['sum', 'mean'])

# Display result
print(result)
```

Output:

Category	sum	mean
A	30	15.0
B	40	20.0
C	65	32.5

Explanation:

- The `groupby('Category')` groups the data by the 'Category' column.
- The `agg(['sum', 'mean'])` applies:
 - `sum` to the 'Values' column, adding up the values for each group.
 - `mean` to the 'Values' column, calculating the average value for each group.

✓ 3. Aggregating Multiple Columns

```
import pandas as pd

# Create a sample dataset
data = {
    'Category': ['A', 'A', 'B', 'B', 'C', 'C'],
    'Values': [10, 20, 15, 25, 30, 35],
    'Score': [85, 90, 78, 88, 92, 95]
}

df = pd.DataFrame(data)

# Aggregation: Applying different functions to 'Values' and 'Score'
result = df.groupby('Category').agg({'Values': 'sum', 'Score': 'mean'})
print(result)
```

Output:

	Values	Score
Category		
A	30	87.5
B	40	83.0
C	65	93.5

Explanation:

1. **Dataset:** The dataset contains three columns:

- **Category:** Groups (A, B, C).
- **Values:** Numeric values associated with each group.
- **Score:** Additional numeric data for each group.

2. **groupby and agg:**

- `groupby('Category')` : Groups the data by the `Category` column.
- `agg({'Values': 'sum', 'Score': 'mean'})` :
 - Applies the `sum` function to the `Values` column for each group.
 - Applies the `mean` function to the `Score` column for each group.

3. **Output:**

- The result shows the aggregated data:
 - The sum of `Values` for each `Category`.
 - The mean of `Score` for each `Category`.

✓ 4. Custom Aggregation Functions

```
import pandas as pd
```

```
# Sample dataset
```

```
data = {  
    'Category': ['A', 'A', 'B', 'B', 'C', 'C'],  
    'Values': [10, 20, 15, 25, 30, 35],  
    'Score': [1, 2, 3, 4, 5, 6]  
}
```

```
# Creating a DataFrame
```

```
df = pd.DataFrame(data)
```

```
# Aggregation: Custom function to calculate range (max - min) of 'Values'
```

```
result = df.groupby('Category')['Values'].agg(lambda x: x.max() - x.min())  
print(result)
```

Code Explanation:

- `groupby('Category')`: Groups the data by the 'Category' column.
- `agg(lambda x: x.max() - x.min())`: Applies a custom aggregation function to the 'Values' column that calculates the range (max value - min value) for each group.

Output:

```
Category
A      10
B      10
C       5
Name: Values, dtype: int64
```

Output Explanation:

- **A:** The range of values for category A is 20 (maximum) - 10 (minimum) = 10.
- **B:** The range of values for category B is 25 (maximum) - 15 (minimum) = 10.
- **C:** The range of values for category C is 35 (maximum) - 30 (minimum) = 5.

✓ 5. Aggregations with Multiple Keys

```
import pandas as pd

# Sample DataFrame
data = {
    'Category': ['A', 'B', 'A', 'C', 'B', 'C'],
    'Score': [1, 2, 1, 3, 2, 3],
    'Values': [10, 15, 20, 30, 25, 35]
}

df = pd.DataFrame(data)

# Aggregation: Group by 'Category' and 'Score', then calculate sum of 'Values'
result = df.groupby(['Category', 'Score'])['Values'].sum()

# Display the result
print(result)
```

Code Explanation:

- `df.groupby(['Category', 'Score'])`: Groups the data by both the Category and Score columns.
- `['Values']`: Selects the Values column for aggregation.
- `.sum()`: For each group (unique combinations of Category and Score), it calculates the sum of Values.

Output:

```
Category  Score
A          1      30
B          2      40
C          3      65
Name: Values, dtype: int64
```

Output Explanation:

- **Category:** The dataset is grouped by the `Category` column, so the results are organized by the categories 'A', 'B', and 'C'.
- **Score:** For each category, the `Score` column also groups the data. For instance, for category 'A', there is one unique score (1), and for category 'B', there is one unique score (2), etc.
- **Sum of 'Values':** The `sum()` function calculates the total of the `Values` column for each unique combination of `Category` and `Score`:
 - For Category 'A' with Score 1, the sum of `Values` is $10 + 20 = 30$.
 - For Category 'B' with Score 2, the sum of `Values` is $15 + 25 = 40$.
 - For Category 'C' with Score 3, the sum of `Values` is $30 + 35 = 65$.

✓ 2.Grouping

a.What is Grouping?

- Grouping is the process of splitting your dataset into groups based on unique values in one or more columns.
- After grouping, you can perform aggregation operations like summing, averaging, or finding maximum/minimum values for each group.

b.Why Use Grouping?

Grouping is a powerful technique that helps organize your data and allows for more focused analysis. Here's why it's useful:

- **Organize Data into Logical Groups**
Grouping helps you categorize your data based on unique values in one or more columns. This makes it easier to understand and compare different segments of your data.
- **Make Operations Easier on Data Subsets**
By grouping your data, you can easily perform operations, such as aggregation or

transformations, on specific subsets. This makes analysis faster and more efficient, especially when working with large datasets.

- Answer Specific Questions

Grouping allows you to answer targeted questions about your data, such as:

- What is the average sales in each region?
- How many products were sold in each category?

Grouping helps you find insights and patterns within smaller segments of data, which might be missed when looking at the overall dataset.

✓ 3.Key Functions for Data Aggregation and Grouping:

Now that we understand data aggregation and grouping, let's dive into the functions used in Pandas to perform these operations.

✓ a. GroupBy

What is GroupBy?

Imagine you have a list of people, and each person belongs to a group. Now, you want to know something about each group — for example, what the average salary is for people in the same department. The groupby function is like a helper that organizes the data into groups based on certain columns (like "Department" or "City") so that you can easily calculate statistics for each group.

Steps in GroupBy

The groupby() function works in three steps:

1. **Split the data:** First, it divides the data into smaller groups based on a column. For example, if you group by "Department", it will create a group for each unique department (like HR, IT, Finance).
2. **Apply a function:** Then, it performs a calculation (like sum, average, count) for each of these groups.
3. **Combine the results:** Finally, it gives you the result in a new form, like a table that shows the calculations for each group.

✓ Example 1: Grouping and Calculating Average Salary

Let's say you have the following data about employees, their departments, and their salaries:

Employee	Department	Salary
Alice	HR	5000
Bob	IT	7000
Charlie	HR	6000
David	IT	8000
Eve	Finance	5500
Frank	Finance	7200

You want to know the average salary for each department. Here's how it works:

Code Example:

```
import pandas as pd # Import the Pandas library

# Step 1: Create a dataset
data = {
    'Department': ['HR', 'IT', 'HR', 'IT', 'Finance', 'Finance'],
    'Employee': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank'],
    'Salary': [5000, 7000, 6000, 8000, 5500, 7200]
}

df = pd.DataFrame(data) # Convert the dataset into a DataFrame

# Step 2: Group by 'Department' and calculate the average salary
grouped_data = df.groupby('Department')['Salary'].mean()

# Step 3: Display the grouped data
print(grouped_data)
```

Line-by-Line Explanation:

- `import pandas as pd`: We import Pandas, a tool for working with data in Python.
- `data = {...}`: This is your data in the form of a dictionary, where each "key" (e.g., 'Department', 'Employee', 'Salary') has a list of values.
- `pd.DataFrame(data)`: Converts the dictionary into a table-like structure called a DataFrame (like a spreadsheet).
- `df.groupby('Department')`: This groups the rows of data by the unique values in the 'Department' column. So, it creates separate groups for HR, IT, and Finance.
- `['Salary'].mean()`: After grouping, we calculate the average salary for each group (department).
- `print(grouped_data)`: Displays the result.

Output:

```
Department
Finance    6350.0
HR          5500.0
```

```
IT          7500.0
Name: Salary, dtype: float64
```

This means that:

- The average salary for Finance is 6350.
- The average salary for HR is 5500.
- The average salary for IT is 7500.

✓ Example 2: Counting Employees in Each Department

Now, suppose you want to know how many employees work in each department.

Code Example:

```
# Count the number of employees in each department
employee_count = df.groupby('Department')['Employee'].count()
print(employee_count)
```

Explanation:

- `df.groupby('Department')` : Groups the data by department.
- `['Employee'].count()` : Counts how many employees are in each department.

Output:

```
Department
Finance    2
HR          2
IT          2
Name: Employee, dtype: int64
```

This means:

- There are 2 employees in the Finance department.
- There are 2 employees in the HR department.
- There are 2 employees in the IT department.

✓ b. Aggregation:

What is Aggregation?

Aggregation in Pandas is the process of applying multiple operations (like sum, mean, count, etc.) on grouped data. After you group data (e.g., by department or city), you often want to

compute summaries like averages, totals, or other statistics. Aggregation helps you do this efficiently.

Steps in Aggregation:

1. **Group the data:** Similar to `groupby()`, you first group the data by one or more columns.
2. **Apply multiple functions:** After grouping, you apply one or more aggregation functions to the grouped data. These functions calculate summaries for each group (e.g., `sum`, `mean`, `max`, etc.).
3. **Combine the results:** The results of the aggregation are combined and returned as a new `DataFrame` or `Series` that shows the statistics for each group.

✓ Example: Multiple Aggregations

Let's say you want to calculate multiple statistics (mean, sum, max, min) for the "Salary" column, grouped by "Department".

Code Example:

```
import pandas as pd # Import the Pandas library

# Step 1: Create a dataset
data = {
    'Department': ['HR', 'IT', 'HR', 'IT', 'Finance', 'Finance'],
    'Employee': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank'],
    'Salary': [5000, 7000, 6000, 8000, 5500, 7200]
}

df = pd.DataFrame(data) # Convert the dataset into a DataFrame

# Step 2: Group by 'Department' and calculate multiple aggregation metrics
aggregated_data = df.groupby('Department')['Salary'].aggregate(['mean', 'sum', 'max', 'mi

# Step 3: Display the aggregated data
print(aggregated_data)
```

Line-by-Line Explanation:

- `import pandas as pd`: We import Pandas, which helps in data analysis.
- `data = {...}`: This is a dictionary representing our data with three columns – 'Department', 'Employee', and 'Salary'.
- `pd.DataFrame(data)`: Converts the dictionary into a `DataFrame` (a table-like structure).
- `df.groupby('Department')`: Groups the data by the 'Department' column.
- `['Salary']`: We select the 'Salary' column, which is where we will apply the aggregation functions.

- `.aggregate(['mean', 'sum', 'max', 'min'])`: This performs multiple aggregations:
 - `mean`: Calculates the average salary for each department.
 - `sum`: Adds up all the salaries in each department.
 - `max`: Finds the highest salary in each department.
 - `min`: Finds the lowest salary in each department.
- `print(aggregated_data)`: Displays the results.

Output:

	mean	sum	max	min
Department				
Finance	6350.0	12700	7200	5500
HR	5500.0	11000	6000	5000
IT	7500.0	15000	8000	7000

This means:

For Finance:

- The average salary is 6350.
- The total salary is 12700.
- The highest salary is 7200.
- The lowest salary is 5500.

For HR:

- The average salary is 5500.
- The total salary is 11000.
- The highest salary is 6000.
- The lowest salary is 5000.

For IT:

- The average salary is 7500.
- The total salary is 15000.
- The highest salary is 8000.
- The lowest salary is 7000.

✓ c.Understanding Mean, Median, Mode and Standard Deviation

What Are These Functions?

These are statistical measures used to summarize and describe a dataset. Let's understand them:

- **Mean** : The average value of a dataset. It's the sum of all values divided by the total number of values.
- **Median** : The middle value in a sorted dataset. If the dataset has an even number of values, it's the average of the two middle values.
- **Mode** : The most frequently occurring value in the dataset.
- **Standard Deviation** : Measures how much the values in a dataset deviate (or spread out) from the mean. A higher standard deviation means the data is more spread out.
- **Variance** : The average of the squared differences from the mean. It represents the spread of the data.

Why Are These Functions Important?

- They provide quick insights into the **central tendency** (mean, median, mode) and the **variability** (standard deviation, variance) of your data.
- These measures help to:
 - Identify trends.
 - Detect anomalies.
 - Make data-driven decisions.

✓ Mean

Definition:

The mean is the average of all values in the dataset. It gives a sense of the central value of the data.

Code Example:

```
import pandas as pd

# Create a dataset
data = {'Scores': [85, 90, 78, 92, 88, 95, 89]}
df = pd.DataFrame(data)

# Calculate the mean
mean_value = df['Scores'].mean()

print("Mean of the Scores:", mean_value)
```

Explanation of the Code:

- `data = {...}` : A dictionary representing scores of students.
- `pd.DataFrame(data)` : Converts the dictionary into a DataFrame for easier manipulation.
- `df['Scores'].mean()` :
 - Accesses the column **Scores**.

- Calculates the mean using the `mean()` method.
- `print()` : Displays the calculated mean.

Output:

Mean of the Scores: 88.14285714285714

Explanation of the Output:

The mean is approximately 88.14, which represents the average score of the students.

✓ Median

Definition:

The median is the middle value when the dataset is sorted. It's less affected by extreme values (outliers) compared to the mean.

Code Example:

```
import pandas as pd

# Create a dataset
data = {'Scores': [85, 90, 78, 92, 88, 95, 89]}
df = pd.DataFrame(data)

# Calculate the median
median_value = df['Scores'].median()

print("Median of the Scores:", median_value)
```

Explanation of the Code

- `data = {...}` : A dictionary containing the scores of students.
- `pd.DataFrame(data)` : Converts the dictionary into a DataFrame for easier manipulation.
- `df['Scores'].median()` :
 - Accesses the Scores column.
 - Calculates the median using the `median()` method.
- `print()` : Displays the calculated median.

Output:

Median of the Scores: 89.0

Explanation of the Output:

The median is **89.0**, which is the middle value of the sorted scores dataset.

✓ Mode

Definition: The mode is the most frequently occurring value in the dataset. A dataset can have no mode, one mode, or multiple modes.

Code Example:

```
import pandas as pd

# Create a dataset
data = {'Scores': [85, 90, 78, 92, 88, 95, 89, 90]}
df = pd.DataFrame(data)

# Calculate the mode
mode_value = df['Scores'].mode()

print("Mode of the Scores:", mode_value)
```

Explanation of the Code:

- `data = {...}`: A dictionary representing scores of students, including a repeated score (90).
- `pd.DataFrame(data)`: Converts the dictionary into a DataFrame for easier manipulation.
- `df['Scores'].mode()`: Calculates the mode of the 'Scores' column, which finds the most frequent value(s).
- `print()`: Displays the calculated mode.

Output:

```
Mode of the Scores: 0    90 dtype: int64
```

Explanation of the Output:

The mode is **90**, as it appears twice in the dataset, while all other scores appear only once. Therefore, 90 is the most frequent score in the dataset.

✓ Standard Deviation

Definition:

The standard deviation measures how much the values in a dataset vary from the mean. A smaller standard deviation indicates that the values are closer to the mean.

Code Example:

```
import pandas as pd

# Create a dataset
data = {'Scores': [85, 90, 78, 92, 88, 95, 89]}
df = pd.DataFrame(data)

# Calculate the standard deviation
std_deviation = df['Scores'].std()

print("Standard Deviation of the Scores:", std_deviation)
```

Explanation of the Code:

- `data = {...}`: A dictionary representing scores of students.
- `pd.DataFrame(data)`: Converts the dictionary into a DataFrame for easier manipulation.
- `df['Scores'].std()`:
 - Accesses the Scores column.
 - Calculates the standard deviation using the `std()` method.
- `print()`: Displays the calculated standard deviation.

Output:

```
Standard Deviation of the Scores: 5.570013400794177
```

Explanation of the Output:

The standard deviation is approximately **5.57**, meaning that most of the scores deviate from the mean by this amount. A smaller standard deviation would indicate that the scores are more clustered around the mean.

✓ Merging, Joining, and Concatenating DataFrames

When you work with data, you often need to combine multiple DataFrames. This is where merging, joining, and concatenating come into play. These operations allow you to combine data from different sources or tables into a single DataFrame, and they can be customized depending on how you want to match the data.

✓ Merging DataFrames

What is Merging?

Merging is the process of combining two or more DataFrames based on common columns. It's similar to SQL joins, where you can specify the type of join you want to perform (like inner, left, right, or outer join).

Merging is used when you have related data in two DataFrames and you want to combine them based on a shared key or column.

Steps for Merging:

1. **Identify common column(s):** Determine the column(s) in both DataFrames that should be used for matching rows.
2. **Choose the merge type:** Decide whether you want to perform an inner, left, right, or outer join.
3. **Merge the DataFrames:** Use the `pd.merge()` function to merge the DataFrames.
4. **Review the result:** After merging, inspect the resulting DataFrame to ensure it contains the data you need.

✓ Types of Merging in Detail

Let's now explore the four types of merging operations in Pandas: Inner, Left, Right, and Outer merges. We'll explain each with examples.

✓ A. Inner Merge

An inner merge only includes the rows with matching values in both DataFrames. Any rows that do not have a matching value in both DataFrames are excluded from the result.

How it works:

- An inner merge includes only the rows where the values in the common column(s) match in both DataFrames.
- If a row in one DataFrame doesn't have a corresponding match in the other, it will not appear in the result.

Syntax:

```
pd.merge(left, right, on='common_column', how='inner')
```

Explanation:

- `left` : First DataFrame
- `right` : Second DataFrame
- `on` : Column to merge on (should be present in both DataFrames)

- how: 'inner' (default value for merge type)

✓ Inner Merge Example

Let's merge two DataFrames based on a shared column (Department_ID).

Code:

Code Example:

```
import pandas as pd

# First DataFrame: Employee details
df_employee = pd.DataFrame({
    'Employee': ['Alice', 'Bob', 'Charlie'],
    'Department_ID': [1, 2, 1],
    'Salary': [5000, 7000, 6000]
})

# Second DataFrame: Department details
df_department = pd.DataFrame({
    'Department_ID': [1, 2, 3],
    'Department_Name': ['HR', 'IT', 'Finance']
})

# Perform an inner merge
merged_data = pd.merge(df_employee, df_department, on='Department_ID', how='inner')

# Display the result
print(merged_data)
```

Explanation of Code:

- `import pandas as pd`: Imports the pandas library, which is essential for data manipulation in Python.
- `df_employee = pd.DataFrame({...})`: Creates the first DataFrame `df_employee` with employee details, including `Employee`, `Department_ID`, and `Salary`.
- `df_department = pd.DataFrame({...})`: Creates the second DataFrame `df_department` with department details, including `Department_ID` and `Department_Name`.
- `pd.merge(df_employee, df_department, on='Department_ID', how='inner')`: Performs the inner merge on the `Department_ID` column. The `how='inner'` parameter ensures that only rows with matching `Department_ID` values in both DataFrames are included.
- `print(inner_merged_data)`: Displays the result of the inner merge.

Output:

Inner Merge Result:

Employee	Department_ID	Salary	Department_Name
Alice	1	5000	HR
Bob	2	7000	IT

0	Alice	1	5000	HR
1	Charlie	1	6000	HR
2	Bob	2	7000	IT

Explanation of Output:

- In this case, the departments **HR** and **IT** are matched with employees. The department **Finance** is excluded because there are no employees with `Department_ID 3`.
- **Alice** and **Charlie** belong to the HR department, with their salaries being 5000 and 6000, respectively.
- **Bob** belongs to the IT department with a salary of 7000.

✓ B. Left Merge

A **left merge** includes all rows from the left DataFrame and only the matching rows from the right DataFrame. If a row in the left DataFrame doesn't have a corresponding match in the right DataFrame, it will still be included, but with `NaN` values for columns from the right DataFrame.

How it works:

- The left merge keeps **all rows** from the left DataFrame.
- If there is no match in the right DataFrame, the result will include the row from the left DataFrame, with missing values (`NaN`) for columns from the right DataFrame.

Syntax:

```
pd.merge(left, right, on='common_column', how='left')
```

✓ Example: Left Merge

Code Example:

```
import pandas as pd # Importing the pandas library

# First DataFrame: Employee details
df_employee = pd.DataFrame({
    'Employee': ['Alice', 'Bob', 'Charlie'],
    'Department_ID': [1, 2, 1],
    'Salary': [5000, 7000, 6000]
})

# Second DataFrame: Department details
df_department = pd.DataFrame({
    'Department_ID': [1, 2, 3],
    'Department_Name': ['HR', 'IT', 'Finance']
})
```

```
# Perform a left merge
left_merged_data = pd.merge(df_employee, df_department, on='Department_ID', how='left')

# Display the result
print(left_merged_data)
```

Explanation of Code:

- `import pandas as pd`: Imports the pandas library, which is essential for data manipulation in Python.
- `df_employee = pd.DataFrame({...})`: Creates the first DataFrame `df_employee` with employee details, including `Employee`, `Department_ID`, and `Salary`.
- `df_department = pd.DataFrame({...})`: Creates the second DataFrame `df_department` with department details, including `Department_ID` and `Department_Name`.
- `pd.merge(df_employee, df_department, on='Department_ID', how='left')`: Performs the left merge on the `Department_ID` column. The `how='left'` parameter ensures that all rows from the left DataFrame (`df_employee`) are included, and only matching rows from the right DataFrame (`df_department`) are added. If there is no match in the right DataFrame, the result will contain `NaN` for the missing values.
- `print(left_merged_data)`: Displays the result of the left merge.

Output:

Left Merge Result:

	Employee	Department_ID	Salary	Department_Name
0	Alice	1	5000	HR
1	Bob	2	7000	IT
2	Charlie	1	6000	HR

In this case:

- The **HR** department and **IT** department match with employees, so their data is included.
- The **Finance** department does not match with any employee in `df_employee`, but all rows from the left DataFrame (`df_employee`) are kept with the department name as `NaN` for unmatched rows.

✓ C.Right Merge

A **right merge** is similar to a **left merge**, but it keeps all rows from the right DataFrame and only the matching rows from the left DataFrame. If a row in the right DataFrame doesn't have a corresponding match in the left DataFrame, the result will include `NaN` values for the columns from the left DataFrame.

How it works:

- Keeps all rows from the **right DataFrame**.
- If there is no match in the **left DataFrame**, the result will include `NaN` for the left DataFrame's columns.

Syntax:

```
pd.merge(left, right, on='common_column', how='right')
```

- `how='right'`: This specifies that the merge should include all rows from the **right DataFrame**, even if there is no matching row in the **left DataFrame**.

✓ Example: Right Merge

Code Example:

```
import pandas as pd # Importing the pandas library

# First DataFrame: Employee details
df_employee = pd.DataFrame({
    'Employee': ['Alice', 'Bob', 'Charlie'],
    'Department_ID': [1, 2, 1],
    'Salary': [5000, 7000, 6000]
})

# Second DataFrame: Department details
df_department = pd.DataFrame({
    'Department_ID': [1, 2, 3],
    'Department_Name': ['HR', 'IT', 'Finance']
})

# Perform a right merge
right_merged_data = pd.merge(df_employee, df_department, on='Department_ID', how='right')

# Display the result
print("Right Merge Result:")
print(right_merged_data)
```

Explanation of Code:

- `import pandas as pd`: Imports the pandas library, which is essential for data manipulation in Python.
- `df_employee = pd.DataFrame({...})`: Creates the first DataFrame `df_employee` with employee details, including `Employee`, `Department_ID`, and `Salary`.
- `df_department = pd.DataFrame({...})`: Creates the second DataFrame `df_department` with department details, including `Department_ID` and `Department_Name`.

- `pd.merge(df_employee, df_department, on='Department_ID', how='right')`: Performs the right merge on the `Department_ID` column. The `how='right'` parameter ensures that all rows from the right DataFrame (`df_department`) are included, and only matching rows from the left DataFrame (`df_employee`) are added. If there is no match in the left DataFrame, the result will contain `NaN` for the missing values from the left DataFrame.
- `print(right_merged_data)`: Displays the result of the right merge.

Output:

Right Merge Result:

	Employee	Department_ID	Salary	Department_Name
0	Alice	1	5000	HR
1	Bob	2	7000	IT
2	Charlie	1	6000	HR
3	NaN	3	NaN	Finance

Explanation of Output:

- Row 1: Alice from the HR department has a salary of 5000.
- Row 2: Bob from the IT department has a salary of 7000.
- Row 3: Charlie from the HR department has a salary of 6000.
- Row 4: The Finance department (Department_ID 3) is included because it's part of the right DataFrame, but there are no matching employees from the left DataFrame. Therefore, the `Employee` and `Salary` columns are `NaN` for this row.

✓ D.Outer Merge

An outer merge includes all rows from both DataFrames, and where there is no match, the missing values are filled with `NaN`.

How it works:

- **Outer merge** includes all rows from both DataFrames.
- Any missing data from either DataFrame will be filled with `NaN`.

Syntax:

```
pd.merge(left, right, on='common_column', how='outer')
```

✓ Example:Outer Merge

Code Example:

```
import pandas as pd # Importing the pandas library

# First DataFrame: Employee details
df_employee = pd.DataFrame({
    'Employee': ['Alice', 'Bob', 'Charlie'],
    'Department_ID': [1, 2, 1],
    'Salary': [5000, 7000, 6000]
})

# Second DataFrame: Department details
df_department = pd.DataFrame({
    'Department_ID': [1, 2, 3],
    'Department_Name': ['HR', 'IT', 'Finance']
})

# Perform an outer merge
outer_merged_data = pd.merge(df_employee, df_department, on='Department_ID', how='outer')

# Display the result
print(outer_merged_data)
```

Output:

Outer Merge Result:

	Employee	Department_ID	Salary	Department_Name
0	Alice	1	5000	HR
1	Bob	2	7000	IT
2	Charlie	1	6000	HR
3	NaN	3	NaN	Finance

Explanation of Output:

- Row 1: Alice from the HR department has a salary of 5000.
- Row 2: Bob from the IT department has a salary of 7000.
- Row 3: Charlie from the HR department has a salary of 6000.
- Row 4: The Finance department (Department_ID 3) is included because it's part of the outer merge, but there are no matching employees from the left DataFrame. Therefore, the Employee and Salary columns are NaN for this row.

✓ Joining DataFrames

What is Joining?

Joining is a method of combining two DataFrames based on an index or column. Unlike merging, which typically merges DataFrames based on a column, joining is more often used when

combining DataFrames based on their indexes. You can join DataFrames on both their indexes and columns, and it provides a way to combine data for analysis.

Steps for Joining:

1. **Identify common index or column:** Determine the index or column(s) in both DataFrames that should be used for matching rows.
2. **Choose the join type:** Decide whether you want to perform a left, right, outer, or inner join.
3. **Join the DataFrames:** Use the `df.join()` method to join the DataFrames based on the index or column.
4. **Review the result:** After joining, inspect the resulting DataFrame to ensure it contains the data you need.

✓ Types of Joining

There are four main types of joins in Pandas:

1. **Left Join**
2. **Right Join**
3. **Outer Join**
4. **Inner Join**

We will go through each type of join with examples, and I will explain each line of code in detail.

✓ 1. Left Join

Definition:

A Left Join keeps all the rows from the left DataFrame, and it adds matching rows from the right DataFrame.

If no match is found, NaN values are added for columns from the right DataFrame.

Syntax:

```
left_joined_data = df1.join(df2, how='left')
```

✓ Example:

Code Example:


```
import pandas as pd # Import pandas library

# First DataFrame: Employee details
df_employee = pd.DataFrame({
    'Employee': ['Alice', 'Bob', 'Charlie'],
    'Department_ID': [1, 2, 1],
    'Salary': [5000, 7000, 6000]
})

# Second DataFrame: Department details
df_department = pd.DataFrame({
    'Department_ID': [1, 2, 3],
    'Department_Name': ['HR', 'IT', 'Finance']
})

# Setting 'Department_ID' as index for both DataFrames
df_employee.set_index('Department_ID', inplace=True)
df_department.set_index('Department_ID', inplace=True)

# Perform a Left Join
left_joined_data = df_employee.join(df_department, how='left')

# Display the result
print("Left Join Result:")
print(left_joined_data)
```

Explanation of Code:

- **import pandas as pd:** We import the pandas library to work with DataFrames.
- **Creating DataFrames:**
 - `df_employee` contains details about employees: Employee, Department_ID, and Salary.
 - `df_department` contains details about departments: Department_ID and Department_Name.
- **Setting the index:**
 - We set the column `Department_ID` as the index for both DataFrames using `set_index()`. This allows us to join the DataFrames based on their `Department_ID` index.
- **`df_employee.join(df_department, how='left')`:** This performs the left join. All rows from the left DataFrame (`df_employee`) are included, and only the matching rows from the right DataFrame (`df_department`) are added. If there is no match, NaN will appear for columns from the right DataFrame.
- **`print(left_joined_data)`:** Displays the result of the left join.

Output:

Left Join Result:

	Employee	Salary	Department_Name
Department_ID			
1	Alice	5000	HR
2	Bob	7000	IT
1	Charlie	6000	HR

Explanation of Output:

- All rows from `df_employee` are included, and matching rows from `df_department` are added.
- Since all employees have matching departments, the `Department_Name` is populated for each row.
- If no match had been found for a particular employee, the `Department_Name` column would have been filled with `NaN`.

✓ 2. Right Join

Definition:

A Right Join keeps all the rows from the right DataFrame and adds matching rows from the left DataFrame.

If no match is found, `NaN` values are added for columns from the left DataFrame.

Syntax:

```
right_joined_data = df1.join(df2, how='right')
```

✓ Example:

Code Example:

```
import pandas as pd # Import pandas library

# First DataFrame: Employee details
df_employee = pd.DataFrame({
    'Employee': ['Alice', 'Bob', 'Charlie'],
    'Department_ID': [1, 2, 1],
    'Salary': [5000, 7000, 6000]
})

# Second DataFrame: Department details
df_department = pd.DataFrame({
    'Department_ID': [1, 2, 3],
    'Department_Name': ['HR', 'IT', 'Finance']
})
```

```
})
```

```
# Setting 'Department_ID' as index for both DataFrames
df_employee.set_index('Department_ID', inplace=True)
df_department.set_index('Department_ID', inplace=True)

# Perform a Right Join
right_joined_data = df_employee.join(df_department, how='right')

# Display the result
print("Right Join Result:")
print(right_joined_data)
```

Explanation of Code:

- `df_employee.join(df_department, how='right')`: This performs the right join. All rows from the right DataFrame (`df_department`) are included, and only matching rows from the left DataFrame (`df_employee`) are added. If no match is found, NaN will appear for columns from the left DataFrame.
- `print(right_joined_data)`: Displays the result of the right join.

Output:

Right Join Result:

	Employee	Salary	Department_Name
Department_ID			
1	Alice	5000	HR
2	Bob	7000	IT
1	Charlie	6000	HR
3	NaN	NaN	Finance

Explanation of Output:

- All rows from `df_department` are included, and matching rows from `df_employee` are added.
- For `Department_ID 3 (Finance)`, there is no matching employee, so `Employee` and `Salary` columns are filled with `NaN`.

✓ 3. Outer Join

Definition:

An **Outer Join** keeps all rows from both the left and right DataFrames, and where there is no match, the missing values are filled with `NaN`.

Syntax:

```
outer_joined_data = df1.join(df2, how='outer')
```

✓ Example:

Code Example:

```
import pandas as pd # Import pandas library

# First DataFrame: Employee details
df_employee = pd.DataFrame({
    'Employee': ['Alice', 'Bob', 'Charlie'],
    'Department_ID': [1, 2, 1],
    'Salary': [5000, 7000, 6000]
})

# Second DataFrame: Department details
df_department = pd.DataFrame({
    'Department_ID': [1, 2, 3],
    'Department_Name': ['HR', 'IT', 'Finance']
})

# Setting 'Department_ID' as index for both DataFrames
df_employee.set_index('Department_ID', inplace=True)
df_department.set_index('Department_ID', inplace=True)

# Perform an Outer Join
outer_joined_data = df_employee.join(df_department, how='outer')

# Display the result
print("Outer Join Result:")
print(outer_joined_data)
```

Explanation of Code:

- `df_employee.set_index('Department_ID', inplace=True)`: Sets the `Department_ID` as the index for the `df_employee` DataFrame.
- `df_department.set_index('Department_ID', inplace=True)`: Sets the `Department_ID` as the index for the `df_department` DataFrame.
- `df_employee.join(df_department, how='outer')`: Performs the **outer join**. All rows from both DataFrames are included, and missing values are filled with `NaN` where there is no match.
- `print(outer_joined_data)`: Displays the result of the outer join.

Output:

Outer Join Result:

```
Employee  Salary  Department_Name
```

Department_ID			
1	Alice	5000	HR
2	Bob	7000	IT
1	Charlie	6000	HR
3	NaN	NaN	Finance

Explanation of Output:

- All rows from both `df_employee` and `df_department` are included in the result.
- For `Department_ID` 3 (Finance), there is no matching employee, so the **Employee** and **Salary** columns contain `NaN`.
- For employees Alice, Bob, and Charlie, their respective departments and salaries are displayed correctly.

✓ 4. Inner Join

Definition:

An Inner Join includes only the rows that have matching values in both DataFrames based on the specified key(s). Any rows without matching values in both DataFrames are excluded from the result.

Syntax:

```
inner_joined_data = df1.join(df2, how='inner')
```

✓ Example:

Code Example:

```
import pandas as pd # Import pandas library

# First DataFrame: Employee details
df_employee = pd.DataFrame({
    'Employee': ['Alice', 'Bob', 'Charlie'],
    'Department_ID': [1, 2, 1],
    'Salary': [5000, 7000, 6000]
})

# Second DataFrame: Department details
df_department = pd.DataFrame({
    'Department_ID': [1, 2, 3],
    'Department_Name': ['HR', 'IT', 'Finance']
})

# Setting 'Department_ID' as index for both DataFrames
```

```
df_employee.set_index('Department_ID', inplace=True)
df_department.set_index('Department_ID', inplace=True)

# Perform an Inner Join
inner_joined_data = df_employee.join(df_department, how='inner')

# Display the result
print("Inner Join Result:")
print(inner_joined_data)
```

Explanation of Code:

- **import pandas as pd:** Imports the pandas library to work with DataFrames.
- **df_employee:** This DataFrame contains employee details such as Employee name, Department ID, and Salary.
- **df_department:** This DataFrame contains department details, including Department ID and Department Name.
- **set_index():** Sets the `Department_ID` as the index for both DataFrames, allowing the join to be performed based on this column.
- **df_employee.join(df_department, how='inner'):** Performs an Inner Join. Only the rows with matching `Department_ID` in both `df_employee` and `df_department` will be included.
- **print(inner_joined_data):** Displays the result of the Inner Join.

Output:

Inner Join Result:

	Employee	Salary	Department_Name
Department_ID			
1	Alice	5000	HR
1	Charlie	6000	HR
2	Bob	7000	IT

Explanation of Output:

- Only the rows with matching `Department_ID` values from both `df_employee` and `df_department` are included.
- The rows with `Department_ID` 3 (Finance) are excluded, as there is no matching employee in the `df_employee` DataFrame.
- Employees Alice and Charlie are in the HR department, and Bob is in the IT department, with their corresponding salaries correctly displayed.

✓ Concatenating DataFrames

✓ What is Concatination ?

Concatenation in Pandas is the process of combining multiple DataFrames along a particular axis (either rows or columns). It is typically used when you want to append rows or columns from one DataFrame to another, rather than merging based on common columns or indexes.

✓ Steps for Concatenating DataFrames

1. Split the DataFrames

- Split the data into two or more DataFrames that you want to concatenate.
- Ensure the data you want to combine is in the correct format.
 - If concatenating along rows, the DataFrames should have the same columns.
 - If concatenating along columns, the DataFrames should have the same index (row labels).
- *Example:**
 df1 and df2 are two DataFrames that we want to concatenate. Each DataFrame contains information about different employees in different departments.

```
import pandas as pd # Import pandas library

# First DataFrame: Employee details in Department 1
df1 = pd.DataFrame({
    'Employee': ['Alice', 'Bob'],
    'Department': ['HR', 'IT'],
    'Salary': [5000, 7000]
})

# Second DataFrame: Employee details in Department 2
df2 = pd.DataFrame({
    'Employee': ['Charlie', 'David'],
    'Department': ['HR', 'Finance'],
    'Salary': [6000, 8000]
})
```

2. Choose the Axis for Concatenation

Choose the axis along which you want to concatenate the DataFrames. There are two options:

- **Axis 0 (default):** Concatenates DataFrames vertically, i.e., by adding rows.
- **Axis 1:** Concatenates DataFrames horizontally, i.e., by adding columns.

Example for Axis 0 (Vertical Concatenation):

We are adding more rows to the existing DataFrame, i.e., stacking df2 below df1.

```
concat_axis0 = pd.concat([df1, df2], axis=0)
```

3. Concatenate the DataFrames

Use `pd.concat()` to concatenate the DataFrames. The method takes a list of DataFrames as its argument, and the `axis` parameter specifies the direction of concatenation.

Example for Axis 0 (Vertical Concatenation):

Here, we concatenate `df1` and `df2` along axis 0 (vertically), meaning `df2`'s rows will be appended below `df1`.

```
concat_axis0 = pd.concat([df1, df2], axis=0)
print(concat_axis0)
```

Example for Axis 1 (Horizontal Concatenation):

If we want to concatenate `df1` and `df2` side by side (horizontally), we would use `axis=1`.

```
concat_axis1 = pd.concat([df1, df2], axis=1)
print(concat_axis1)
```

4. Reset the Index (Optional)

After concatenation, if you are appending rows (`axis=0`), the resulting DataFrame may have duplicate or misaligned indexes. You can use `reset_index()` to reset the index to a default integer index.

Example for Resetting Index after Vertical Concatenation:

```
concat_axis0_reset = concat_axis0.reset_index(drop=True)
print(concat_axis0_reset)
```

5. Handle Missing Data (Optional)

When concatenating DataFrames with different columns, missing values will appear for columns that do not match. You can use the `fillna()` method to handle these missing values.

Example for Handling Missing Values:

```
concat_axis1.fillna('Unknown', inplace=True)
print(concat_axis1)
```

Code Example: Vertical and Horizontal Concatenation


```
import pandas as pd # Import pandas library

# First DataFrame: Employee details in Department 1
df1 = pd.DataFrame({
    'Employee': ['Alice', 'Bob'],
    'Department': ['HR', 'IT'],
    'Salary': [5000, 7000]
})

# Second DataFrame: Employee details in Department 2
df2 = pd.DataFrame({
    'Employee': ['Charlie', 'David'],
    'Department': ['HR', 'Finance'],
    'Salary': [6000, 8000]
})

# Concatenate vertically (along rows) - Axis 0
concat_axis0 = pd.concat([df1, df2], axis=0)
print("Concatenation along Axis 0 (Vertical):")
print(concat_axis0)

# Concatenate horizontally (along columns) - Axis 1
concat_axis1 = pd.concat([df1, df2], axis=1)
print("\nConcatenation along Axis 1 (Horizontal):")
print(concat_axis1)

# Reset index for vertical concatenation
concat_axis0_reset = concat_axis0.reset_index(drop=True)
print("\nVertical Concatenation with Reset Index:")
print(concat_axis0_reset)

# Handle missing values in horizontal concatenation (optional)
concat_axis1.fillna('Unknown', inplace=True)
print("\nHorizontal Concatenation with Missing Values Handled:")
print(concat_axis1)
```

Output:

Concatenation along Axis 0 (Vertical):

	Employee	Department	Salary
0	Alice	HR	5000
1	Bob	IT	7000
0	Charlie	HR	6000
1	David	Finance	8000

Concatenation along Axis 1 (Horizontal):

	Employee	Department	Salary	Employee	Department	Salary
0	Alice	HR	5000	Charlie	HR	6000

1	Bob	IT	7000	David	Finance	8000
---	-----	----	------	-------	---------	------

```
# Vertical Concatenation with Reset Index:
```

	Employee	Department	Salary
0	Alice	HR	5000
1	Bob	IT	7000
2	Charlie	HR	6000
3	David	Finance	8000

```
# Horizontal Concatenation with Missing Values Handled:
```

	Employee	Department	Salary	Employee	Department	Salary
0	Alice	HR	5000	Charlie	HR	6000
1	Bob	IT	7000	David	Finance	8000

✓ How to Work with Dates and Times in Pandas?

✓ 1. Converting Strings to DateTime

Dates and times often come as text in a dataset (known as strings). To work with them effectively in Pandas, we need to convert them into DateTime objects.

Why Convert?

This conversion makes it easier to sort the data, compare dates, and perform other time-based calculations.

Steps for Converting Strings to DateTime

1. Import Pandas:

Import the Pandas library to use its functions.

2. Use `pd.to_datetime()`:

This function converts columns containing date or time as text into DateTime objects.

Code Example: Converting String to DateTime

```
import pandas as pd

# Sample data with date as string
data = {'Date': ['2024-01-01', '2024-01-02', '2024-01-03'],
        'Sales': [200, 220, 250]}

df = pd.DataFrame(data)

# Convert the 'Date' column from string to datetime
```

```
df['Date'] = pd.to_datetime(df['Date'])

# Display the dataframe
print(df)
```

Code Explanation:

- **Data:** The data dictionary contains a column 'Date' where dates are written as text (strings).
- **Conversion:** `pd.to_datetime(df['Date'])` converts the 'Date' column from text to actual `DateTime` format, which allows us to perform time-related operations.
- **Result:** After conversion, we can now work with the dates in a way that allows for sorting, filtering, and time-based calculations.

Output

	Date	Sales
0	2024-01-01	200
1	2024-01-02	220
2	2024-01-03	250

✓ 2.Setting DateTime as Index

Why Set DateTime as Index?

When you set a `DateTime` column as the index, it makes it easier to perform time-based operations like filtering for specific time periods or resampling the data.

Steps for Setting DateTime as Index

1. **Convert the Date Column** (as shown in the previous example).
2. **Set DateTime Column as Index:** Use the `.set_index()` function to set the 'Date' column as the index of the `DataFrame`.

Code Example: Setting DateTime as Index

```
# Set the 'Date' column as the index
df.set_index('Date', inplace=True)

# Display the updated dataframe
print(df)
```

Code Explanation

- `set_index('Date')`: This makes the 'Date' column the index of the `DataFrame`, so every row is labeled by a specific date.

- `inplace=True` : This means the change will happen directly in the existing DataFrame, and you don't need to create a new one.

Output

	Sales
Date	
2024-01-01	200
2024-01-02	220
2024-01-03	250

✓ Opening and Saving Files in Pandas

✓ Introduction

- In the real world, data is stored in various file formats such as **CSV**, **Excel**, **JSON**, **HTML**, etc.
- **Pandas** makes it very easy to:
 - Read data from these files.
 - Save processed data back into the same or a different format.

✓ 1.Opening and Saving CSV Files

What is a CSV File?

- **CSV** stands for **Comma-Separated Values**.
- It's a simple text file where:
 - Each row represents a record.
 - Columns are separated by commas.
- **CSV** is one of the most commonly used file formats in data analysis.

Why Use CSV Files?

- CSV files are **lightweight** and can be opened in:
 - Any text editor.
 - Spreadsheet software like **Microsoft Excel**.

✓ Opening a CSV File

Code Example:

```
import pandas as pd # Import Pandas library

# Reading data from a CSV file
df = pd.read_csv('data.csv') # Replace 'data.csv' with your file name
print(df.head()) # Displays the first 5 rows of the DataFrame
```

Explanation:

- `import pandas as pd`: Imports the Pandas library.
- `pd.read_csv('data.csv')`: Loads the CSV file into a Pandas DataFrame.
- `df.head()`: Displays the first 5 rows to give you a quick look at the data.

✓ Saving a CSV File

Code Example:

```
# Saving the DataFrame to a new CSV file
df.to_csv('output.csv', index=False)
```

Explanation:

- `df.to_csv('output.csv')`: Saves the DataFrame to a new file named `output.csv`.
- `index=False`: Excludes the row index numbers from the file.

✓ 2. Opening and Saving Excel Files

What is an Excel File?

- Excel files (extensions `.xls` or `.xlsx`) are spreadsheet files widely used for:
 - Storing tabular data
 - Applying formulas
 - Formatting cells
 - Managing multiple sheets within a single file

Why Use Excel Files?

- Commonly used in businesses for:
 - Data entry
 - Reporting
 - Sharing datasets
 - Easily viewable and editable in tools like Microsoft Excel and Google Sheets.
-

✓ Opening an Excel File

Code Example:

```
# Reading data from an Excel file
import pandas as pd # Importing the Pandas library

df = pd.read_excel('data.xlsx') # Replace 'data.xlsx' with your file name
print(df.head()) # Displays the first few rows of the DataFrame
```

Explanation:

- `pd.read_excel('data.xlsx')` : Reads the Excel file into a Pandas DataFrame.
- `df.head()` : Displays the first 5 rows of the DataFrame for a quick preview.
- You can specify the sheet name using the `sheet_name='Sheet1'` parameter if the file contains multiple sheets.

Saving an Excel File

Code Example:

```
# Saving the DataFrame to an Excel file
df.to_excel('output.xlsx', index=False, sheet_name='Summary')
```

Explanation:

- `df.to_excel('output.xlsx')` : Saves the DataFrame to an Excel file named `output.xlsx`.
- `index=False` : Excludes row indices from the saved file.
- `sheet_name='Summary'` : Specifies the name of the sheet where the data will be saved.

✓ 3. Opening and Saving JSON Files

What is a JSON File?

- **JSON** stands for **JavaScript Object Notation**.
- It is a lightweight format for storing structured data, often used in web development and APIs.

Why Use JSON Files?

- They are human-readable and easy for machines to parse, making them ideal for data exchange.

Opening a JSON File

Code Example:

```
# Reading data from a JSON file
df = pd.read_json('data.json') # Replace 'data.json' with your file name
print(df.head())
```

Explanation:

- `pd.read_json('data.json')` : Reads the JSON file into a DataFrame. This command loads the data from the `data.json` file into a Pandas DataFrame, allowing for easy data manipulation and analysis.
- `print(df.head())` : Displays the first few rows of the DataFrame to get a preview of the data.

Saving a JSON File

Code Example:

```
# Saving the DataFrame to a JSON file
df.to_json('output.json', orient='records')
```

Explanation:

- `df.to_json('output.json')` : Saves the DataFrame to a JSON file named `output.json`.
- `orient='records'` : Specifies the format of the saved data as a list of dictionaries, where each dictionary corresponds to a row in the DataFrame.

✓ 4. Opening and Saving HTML Files

What is an HTML File?

HTML stands for HyperText Markup Language and is used for creating web pages.

Pandas can work with HTML tables, either by extracting data from a webpage or by saving a DataFrame as an HTML table.

Why Use HTML Files?

HTML is used when you want to present your data on a web page.

Opening an HTML File

Code Example:

```
# Reading data from an HTML file
url = 'https://example.com/table.html' # Replace with a file path or URL
df_list = pd.read_html(url) # Reads all tables into a list of DataFrames
print(df_list[0]) # Access the first table
```

Explanation:

- `pd.read_html(url)` : Extracts all tables from the HTML page into a list of DataFrames.
- `df_list[0]` : Accesses the first table from the list.

Saving an HTML File

Code Example:

```
# Saving the DataFrame to an HTML file
df.to_html('output.html', index=False)
```

Explanation:

- `df.to_html('output.html')` : Saves the DataFrame as an HTML table.

THANK YOU



Stay updated with us!



[Vishwa Mohan](#)



[Vishwa Mohan](#)



[vishwa.mohan.singh](#)



[Vishwa Mohan](#)